Exercice 4 Report

6.1.2 Questions

1. Learning Rate

Experiment: The network was trained with learning rates of 1.0, 0.01, 0.001, and 0.00001. The validation loss for each epoch is plotted in Figure

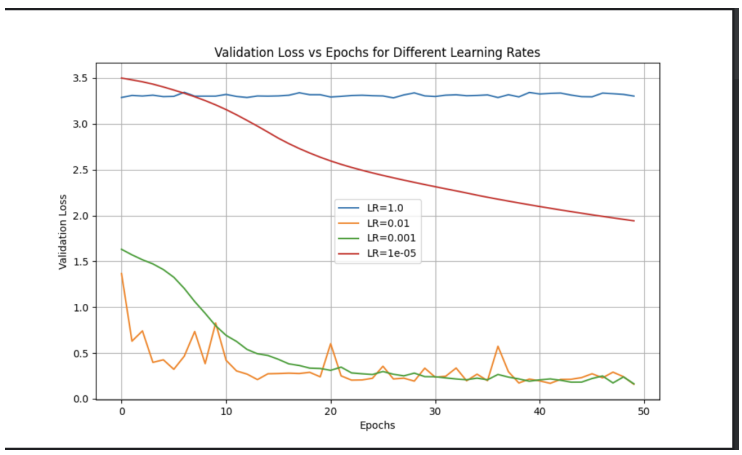


Figure 1: Validation loss vs. epochs for different learning rates.

Analysis:

- **High Learning Rate:** A too-high learning rate (e.g., 1.0) causes the model to diverge or oscillate significantly without converging, as large steps overshoot the optimal point.
- **Low Learning Rate:** A too-low learning rate (e.g., 0.00001) leads to very slow convergence, requiring many more epochs to approach the minimum loss.
- **Optimal Learning Rate:** Rates like 0.01 and 0.001 demonstrate steady convergence and lower validation loss.

2. Epochs

Experiment: The network was trained for 100 epochs. The training, validation, and test losses are plotted in Figure.

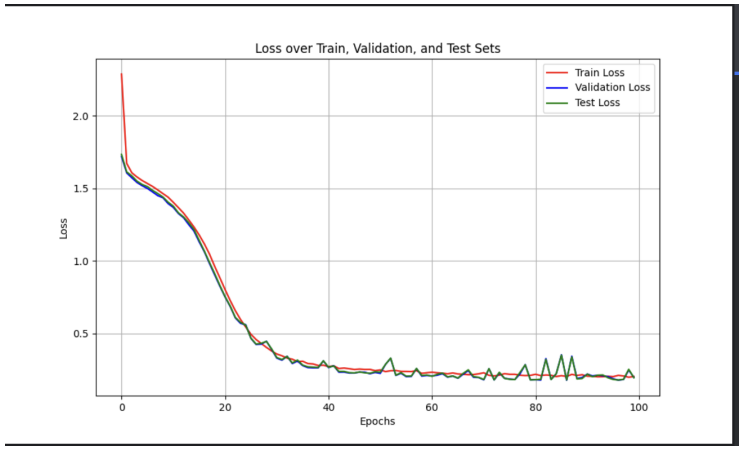
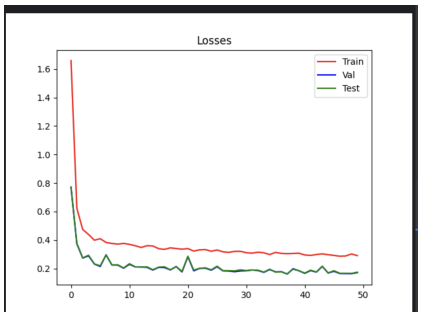


Figure 2: Loss vs. epochs for train, test, and validation sets.

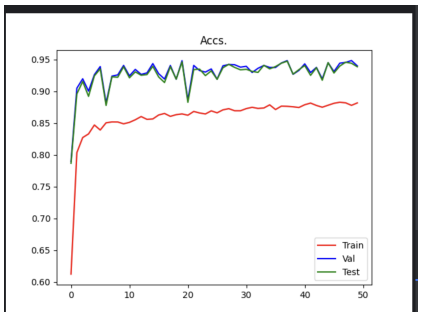
Impact of Number of Epochs on Training: The graph illustrates that initially, the training, validation, and test losses decrease significantly as the network learns and optimizes its weights, indicating that training with more epochs allows the model to better fit the data. However, after approximately 60 epochs, the validation and test losses start to fluctuate or slightly increase, while the training loss continues to decrease. This while be overfitting, where the model becomes too closed to the training data and fails to generalize well. Conversely, training with too few epochs leads to underfitting, as the network does not have enough time to effectively adjust its weights. Reflecting the model's inability to capture the underlying patterns in the data. Thus, finding the right balance in the number of epochs is crucial to avoid both underfitting and overfitting.

3. Batch Normalization

Experiment: A batch normalization layer (nn.BatchNorm1d) was added after each hidden layer. The results were compared with the original model.

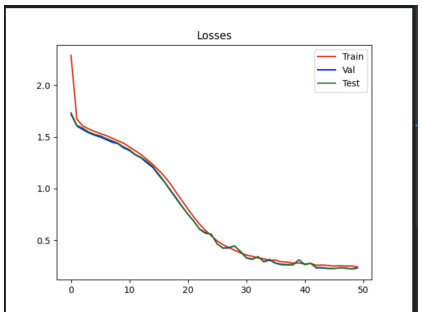


(a) Validation Loss (With Batch Norm)

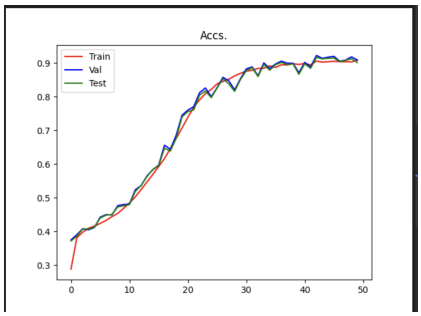


(b) Validation Accuracy (With Batch Norm)

Figure 3: Loss and Accuracy comparison witht batch normalization.



(a) Validation Loss (Example)



(b) Validation Accuracy (Example)

Figure 4: Loss and Accuracy for Example Model.

Effect of Batch Normalization: The batch-normalized model converges faster and achieves better accuracy compared to the regular model. This is primarily due to the fact that batch normalization stabilizes the gradients during training, allowing the model to learn more efficiently. Additionally, the batch-normalized model maintains stable performance over time, while the regular model improves more slowly and gradually. The batch-normalized model is also able to generalize better, as evidenced by its higher test accuracy compared to the training accuracy, indicating that it has learned well from the data.

The key advantage of batch normalization is that it normalizes each layer's input, reducing the internal covariate shift, which means that the distribution of inputs to each layer remains more consistent during training. This reduction in covariate shift makes the model less sensitive to weight initialization and learning rates, thus accelerating the training process. Furthermore, batch normalization acts as a form of regularization by introducing slight noise during training. This helps prevent overfitting and improves the model's ability to generalize to unseen data, making it more robust in real-world applications.

4. Batch Size

Experiment: The network was trained with batch sizes of 1, 16, 128, and 1024. The number of epochs was set to 1, 10, 50, and 50 respectively. The effects on validation accuracy, training speed, and stability were analyzed.

(i) Accuracy: The validation accuracy for each batch size is plotted in Figure

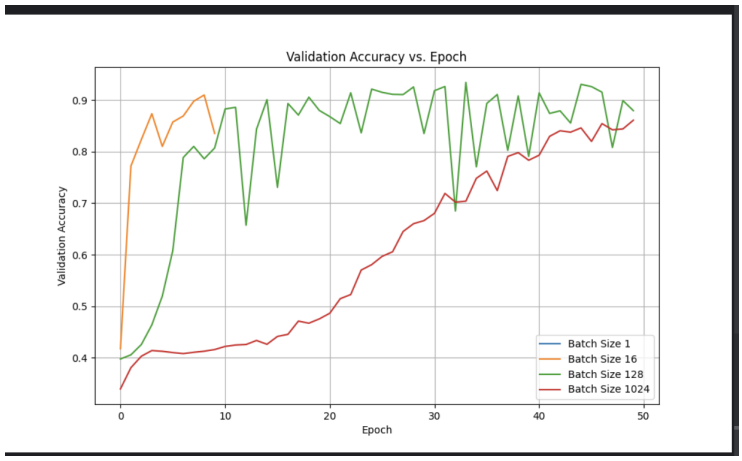


Figure 5: Validation accuracy vs. epochs for different batch sizes.

Analysis: - A very small batch size (e.g., 1) produces noisy gradients and fluctuating accuracy. While it may capture fine details, the instability can hinder consistent learning. - Medium batch sizes (e.g., 16, 128) strike a balance between stability and efficient gradient updates, resulting in better convergence and higher accuracy. - A large batch size (e.g., 1024) converges more slowly and may prematurely flatten the accuracy, reflecting fewer gradient updates and limited exploration of the loss landscape.

(ii) Speed: The number of iterations per epoch is inversely proportional to the batch size and is calculated as:

$$\text{Iterations per epoch} = \frac{\text{Dataset size}}{\text{Batch size}}$$

Figure shows the iteration per epoch for different batch sizes.

```
Batch size: 1, Iterations per epoch: 106897
Batch size: 16, Iterations per epoch: 6681
Batch size: 128, Iterations per epoch: 835
Batch size: 1024, Iterations per epoch: 104
```

Figure 6: Training speed: Number of iterations per epoch for different batch sizes.

Analysis: - Smaller batch sizes require more iterations per epoch, increasing computational time per epoch. - Larger batch sizes reduce the number of iterations, speeding up each epoch. However, this may lead to slower convergence overall due to limited gradient updates.

(iii) Stability: The training loss for each batch size is plotted in Figure

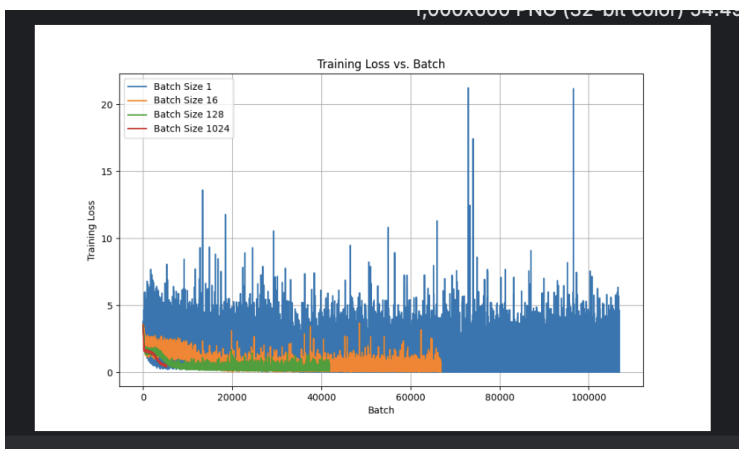


Figure 7: Training loss vs. batch size over epochs.

Analysis: - A batch size of 1 shows high noise in the training loss, as gradients are computed for individual samples, leading to unstable updates. - Medium batch sizes (16, 128) smooth out the noise, resulting in more stable and efficient training. - A batch size of 1024 produces the most stable loss, but the training process may flatten prematurely, limiting convergence quality.

6.2.1 Questions

1. Model with the Best Validation Accuracy: The model with the best validation accuracy demonstrated a significant decrease in training, validation, and test losses throughout the training iterations, as shown in Figure. The prediction space, visualized alongside the losses, shows that the test points align

well with the predicted regions, indicating that the model successfully captured the underlying patterns in the data.

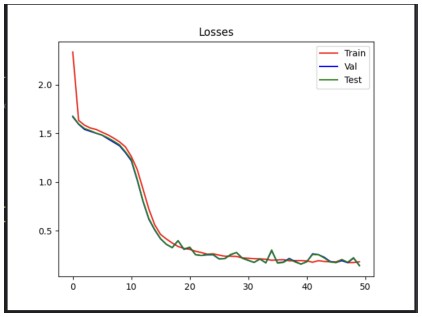


Figure 8: Losses for the model with the best validation accuracy.

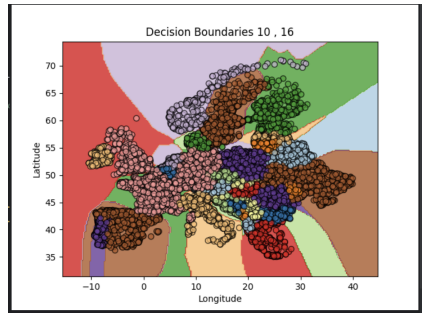


Figure 9: Prediction space for the model with the best accuracy.

2. Model with the Worst Validation Accuracy: The model with the worst validation accuracy exhibited consistently high training, validation, and test losses, as shown in Figure. The prediction space, visualized alongside the losses, highlights poor alignment between the test points and predicted regions. This model under-fit the dataset, as it failed to capture the underlying structure of the data, likely due to insufficient capacity or ineffective training.

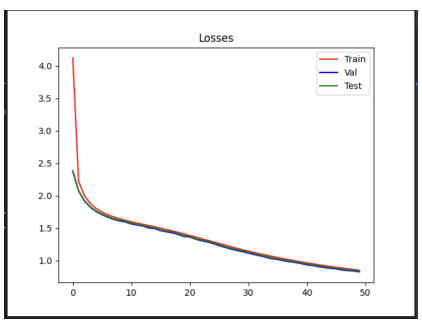


Figure 10: Losses for the model with the worst validation accuracy.

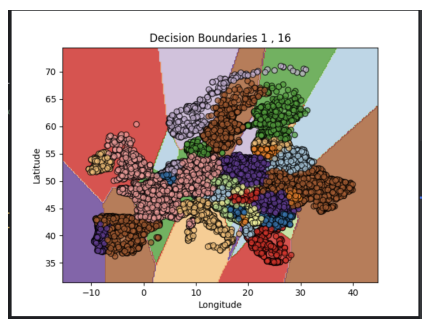


Figure 11: Prediction space for the model with the worst accuracy.

3. Depth of Network

Experiment: Using only the MLPs with a fixed width of 16 neurons per layer, the training, validation, and test accuracies were plotted against the number of hidden layers. The results are shown in Figure

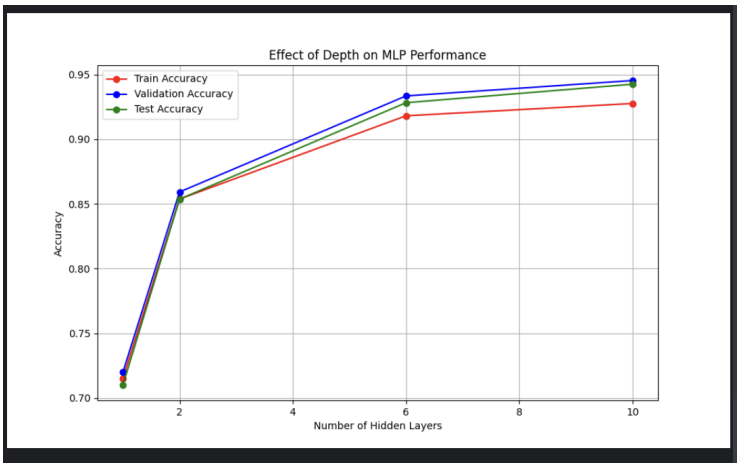


Figure 12: Training, validation, and test accuracy vs. number of hidden layers.

Analysis: The graph shows that increasing the number of hidden layers improves the accuracy across training, validation, and test datasets. This prove that deeper networks have more representational power, allowing them to learn complex patterns in the data (in our case were the deep is not to hight). However, in general, increasing the number of layers comes with several challenges:

- **Overfitting:** A very deep network may overfit the training data, resulting in reduced generalization to unseen data.
- **Vanishing/Exploding Gradient Problem:** As the network becomes deeper, the gradients during backpropagation can diminish/increase, making it harder for the weights in earlier layers to update effectively.
- **Computational Complexity:** Deeper networks require more computation, leading to increased training time and resource consumption.

4. Width of Network

Experiment: Using only the MLPs with a fixed depth of 6 hidden layers, the training, validation, and test accuracies were plotted against the number of neurons in each layer. The results are shown in Figure .

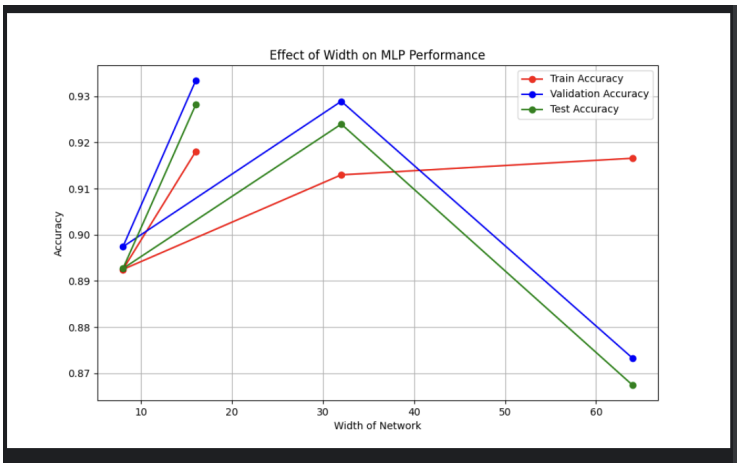


Figure 13: Training, validation, and test accuracy vs. number of neurons per hidden layer.

Analysis: The graph indicates that increasing the number of neurons per hidden layer initially improves the model's accuracy. However, after a certain point, the accuracy plateaus and then begins to decline due to overfitting:

- **Initial Increase:** Adding more neurons allows the network to model more complex patterns, leading to better performance.
- **Plateau:** Beyond a certain width, additional neurons do not significantly contribute to learning, as the model already has sufficient capacity to capture the data's structure.
- **Overfitting:** Excessive width causes the network to memorize the training data rather than generalizing to unseen data, leading to a decline in validation and test accuracy.

5. Monitoring Gradients

Experiment: A network with 100 hidden layers, each containing 4 neurons, was trained for 10 epochs.

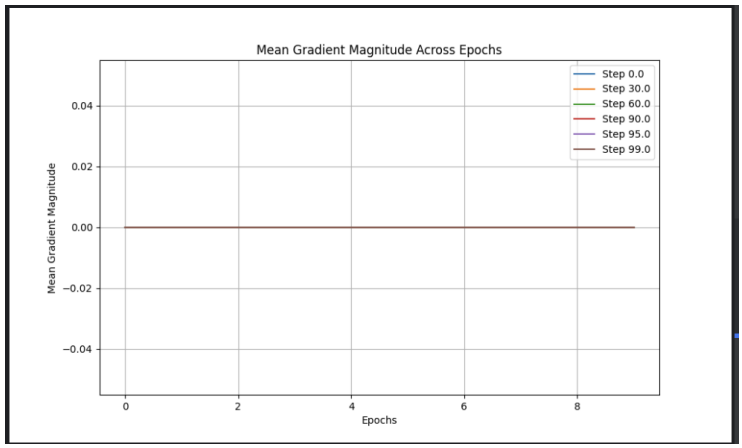


Figure 14: Mean gradient magnitudes vs. training epochs for selected layers (0, 30, 60, 90, 95, 99).

Analysis: The plot indicates that the gradient magnitudes for all layers are effectively zero throughout training. This is a clear sign of the **vanishing gradient problem**, where gradients become too small to meaningfully update the weights during backpropagation. This phenomenon often occurs in very deep networks, as their derivatives tend to shrink the gradients.

Solutions: To mitigate the vanishing gradient problem without reducing the network depth, the following techniques can be employed:

- **Batch Normalization:** Normalizing the inputs of each layer helps maintain stable gradient magnitudes and speeds up training.
- **Residual Connections (ResNet):** Adding skip connections between layers allows gradients to flow directly to earlier layers, alleviating the vanishing gradient issue.

6. Bonus

Experiment: I chose to address the vanishing gradient problem observed in Q5 by implementing batch normalization after each layer in the network.

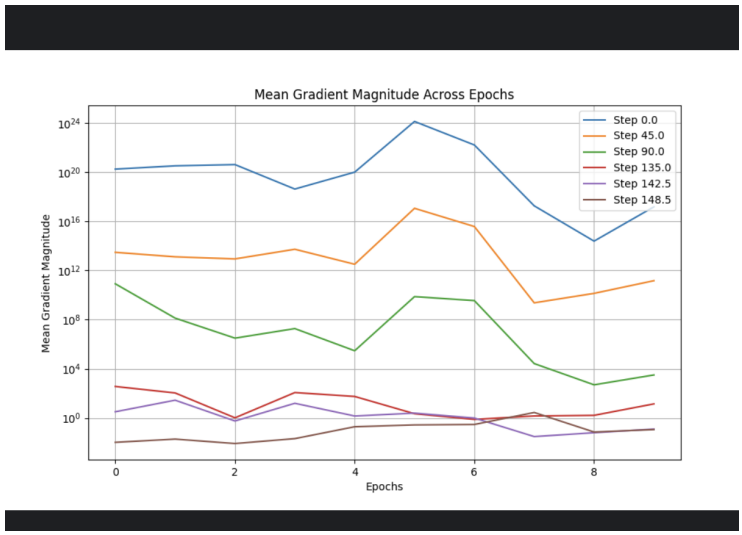


Figure 15: Mean gradient magnitudes (log scale) vs. training epochs for selected layers with batch normalization.

Analysis: The batch normalization technique I implemented effectively mitigated the vanishing gradient problem. The gradient magnitudes remained stable across all layers and epochs, as evidenced by their non-zero values in the log-scale plot. This improvement occurs because batch normalization. By normalizing the inputs to each layer, activations remain within a range that prevents gradients from shrinking excessively during backpropagation.

7.6.1 Best and Worst Models Analysis

Models and Learning Rates: The models evaluated include XGBoost, ResNet18 (Scratch), ResNet18 (Frozen Linear Probing), and ResNet18 (Fine-Tuning) with learning rates of 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, and 10^{-5}.

Results Summary:

Model	Best LR	Second LR	Worst LR
XGBoost	acc: 0.7350		
ResNet18 (Scratch)	10^{-3} (acc: 0.53)	10^{-2} (acc: 0.52)	10^{-4} (acc: 0.49)
ResNet18 (Frozen Linear Probing)	10^{-2} (acc: 0.73)	10^{-1} (acc: 0.71)	10^{-4} (acc: 0.45)
ResNet18 (Fine-Tuning)	10^{-4} (acc: 0.85)	10^{-3} (acc: 0.8)	10^{-1} (acc: 0.5)

Trends Analysis:

- **ResNet18 (Scratch):** This model performed the worst, with an average accuracy of 0.5. Generally, the performance is quite similar across different learning rates, with only small variations. Mid-range learning rates tend to work best, showing neither slow convergence nor failure to converge. However, the model struggles to generalize well due to issues such as insufficient optimization during training.

- **ResNet18 (Fine-Tuning):** Fine-tuning performed better, with the learning rate playing a significant role. Lower learning rates yielded better performance, as they allow for more stable and refined adjustments to the weights during training. This approach works best for transfer learning, where the pre-trained network provides a good starting point for fine-tuning on the target dataset.

- **ResNet18 (Frozen Linear Probing):** In this case, higher learning rates worked better. Linear probing involves freezing the majority of the model and training only the final classification layers. Higher learning rates help in making faster updates to these layers, leading to improved performance, especially when the pre-trained features are already robust.

- **XGBoost:** XGBoost performed well with an accuracy of 0.735. XGBoost typically requires less tuning and can handle overfitting better than deep neural networks when applied to structured datasets.

2. Misclassified Samples Comparison

In this plot, we visualize five samples that were correctly classified by the best baseline but misclassified by the worst baseline. (sorry for the zoom i use all my units on google collab).

Figure 16: Five samples correctly classified by the best baseline but misclassified by the worst baseline.

Ethics

For the completion of this report, I used ChatGPT to assist in generating various plots, refining my LaTeX code, and improving my English writing skills. ChatGPT was also instrumental in helping me understand certain concepts related to the exercises, ensuring a deeper comprehension of the material. Also help me to write this sentence :-).